Exercício 4.1: Deployment com 4 réplicas, rolling update controlado e leitura do histórico

Aluno: Luiz Brito · **Data:** 19/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster local kind v0.32.0 com 3 nós (1 control-plane + 2 workers), Kubernetes v1.36.1, CNI Calico v3.33.0, contexto `kind-fiap-store`, namespace `store-dev`.
- Imagens `store-api:1.0` e `store-api:2.0` construídas a partir de `app/store-api` e carregadas nos nós com `kind load docker-image`.
- Manifesto em `01-deployment.yaml` e sequência exata de comandos em `02-commands.sh`.

Manifesto

```
# Quantas revisoes o rollout history mostra depois de 4 aplicacoes?
# Cada mudanca no template do Pod (imagem, env, limits) cria uma ReplicaSet
# nova e a anterior fica com 0 replicas para permitir rollback. O historico
# e a lista dessas ReplicaSets.
# revisionHistoryLimit conta so as ReplicaSets ANTIGAS (doc oficial:
# "the number of old ReplicaSets to retain"). Com 3, ficam 3 antigas + a
# atual = 4 revisoes visiveis, e nao 3 como o enunciado sugere.
# Para ver exatamente 3 revisoes, o limite tem que ser 2: ai a ReplicaSet
# da revisao 1 e coletada pelo garbage collector e some do historico.
apiVersion: v1
kind: Namespace
metadata:
  name: store-dev
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-dev
```

```yaml
  labels:
    app: store-api
spec:
  replicas: 4
  revisionHistoryLimit: 3
  progressDeadlineSeconds: 120
  minReadySeconds: 15
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: store-api
  template:
    metadata:
      labels:
        app: store-api
    spec:
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          env:
            - name: APP_ENV
              value: dev
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 256Mi
```

Sequência de comandos

```bash
#!/usr/bin/env bash
set -euo pipefail
NS="-n store-dev"
D=deployment/store-api

# revisao 1
kubectl apply -f 01-deployment.yaml
kubectl $NS rollout status $D
kubectl $NS annotate $D kubernetes.io/change-cause="rev1: store-api:1.0, 4 replicas" --overwrite

# (a) imagem 2.0, com kubectl get pods -w em outro terminal
kubectl $NS set image $D store-api=store-api:2.0
kubectl $NS annotate $D kubernetes.io/change-cause="rev2: imagem store-api:2.0" --overwrite
kubectl $NS rollout status $D

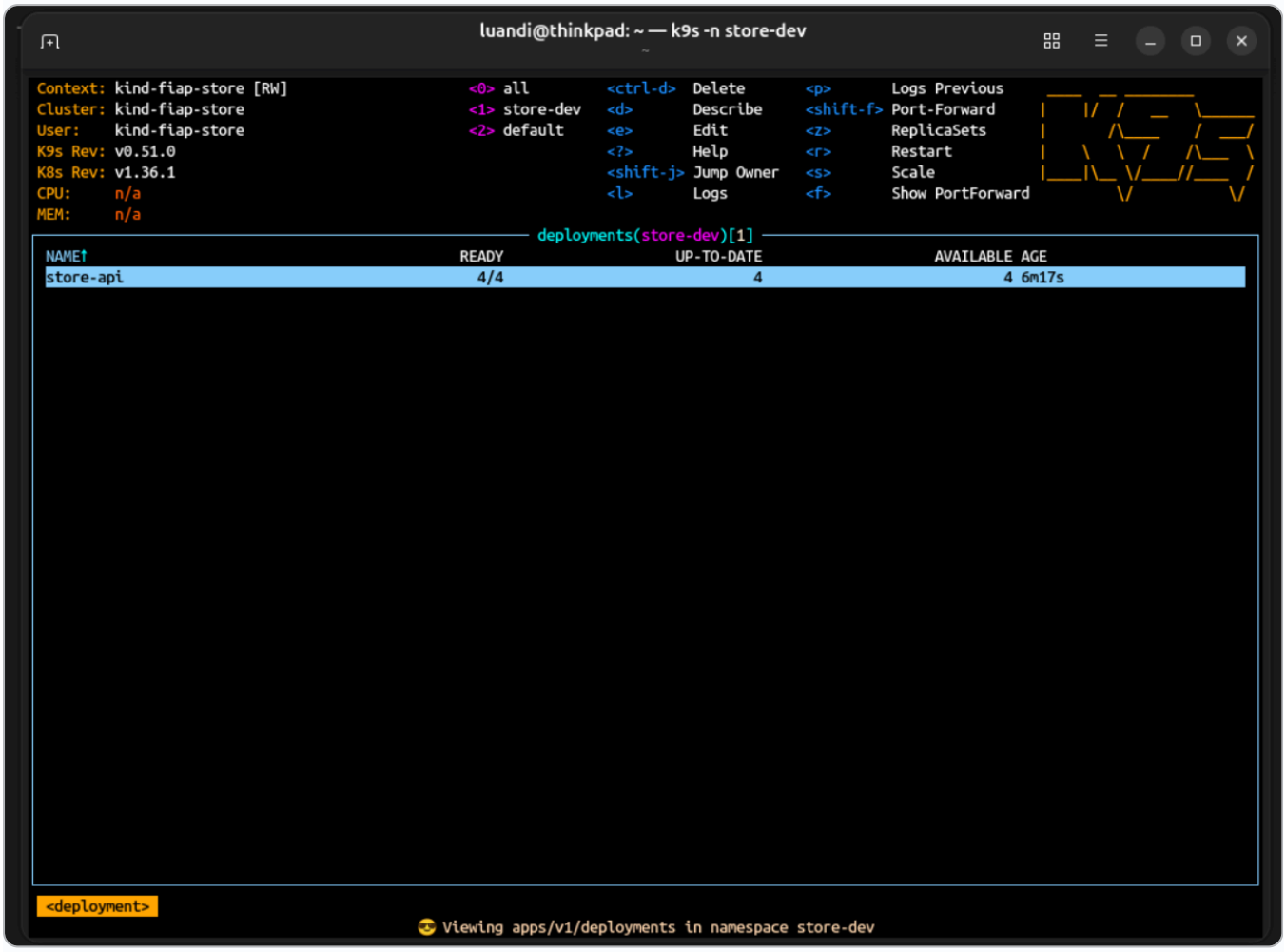
# (b) APP_ENV dev → homolog
kubectl $NS set env $D APP_ENV=homolog
kubectl $NS annotate $D kubernetes.io/change-cause="rev3: APP_ENV=homolog" --overwrite
kubectl $NS rollout status $D

# (c) limits.memory 384Mi
kubectl $NS set resources $D -c store-api --limits=memory=384Mi
kubectl $NS annotate $D kubernetes.io/change-cause="rev4: limits.memory=384Mi" --overwrite
kubectl $NS rollout status $D

kubectl $NS get deploy store-api
kubectl $NS rollout history $D
kubectl $NS get rs -o wide

# prova: com limite 2 sobram 2 antigas + a atual = 3 revisoes
kubectl $NS patch $D -p '{"spec":{"revisionHistoryLimit":2}}'
kubectl $NS rollout history $D
kubectl $NS get rs -o wide
```

Revisão 1: Deployment estável



Item (a): contagem máxima de Pods durante o rollout para 2.0

Saída de `kubectl -n store-dev get pods -w` em outro terminal durante o item (a) (linhas repetidas de um mesmo estado omitidas):

```
NAME                          READY   STATUS              RESTARTS   AGE
store-api-5b5d55bb94-2cqxb    1/1     Running             0          23s
store-api-5b5d55bb94-9pqk8    1/1     Running             0          23s
store-api-5b5d55bb94-kjj7c    1/1     Running             0          23s
store-api-5b5d55bb94-n9h5c    1/1     Running             0          23s
store-api-7948549dc9-hjx4h    0/1     ContainerCreating   0          0s
store-api-7948549dc9-hjx4h    0/1     Running             0          1s
store-api-7948549dc9-hjx4h    1/1     Running             0          7s
store-api-5b5d55bb94-9pqk8    1/1     Terminating         0          45s
store-api-7948549dc9-kfcpz    0/1     ContainerCreating   0          0s
store-api-5b5d55bb94-9pqk8    0/1     Completed           0          45s
store-api-7948549dc9-kfcpz    0/1     Running             0          1s
store-api-7948549dc9-kfcpz    1/1     Running             0          7s
store-api-5b5d55bb94-kjj7c    1/1     Terminating         0          67s
store-api-7948549dc9-jfg8l    0/1     ContainerCreating   0          0s
store-api-5b5d55bb94-kjj7c    0/1     Completed           0          67s
store-api-7948549dc9-jfg8l    0/1     Running             0          1s
store-api-7948549dc9-jfg8l    1/1     Running             0          6s
store-api-5b5d55bb94-2cqxb    1/1     Terminating         0          88s
store-api-7948549dc9-lbgh6    0/1     ContainerCreating   0          0s
store-api-5b5d55bb94-2cqxb    0/1     Completed           0          88s
store-api-7948549dc9-lbgh6    0/1     Running             0          1s
store-api-7948549dc9-lbgh6    1/1     Running             0          6s
store-api-5b5d55bb94-n9h5c    1/1     Terminating         0          109s
```

Contagem máxima observada: 5 Pods simultâneos (4 antigos + 1 novo). Com `maxSurge: 1` o controller só cria 1 Pod acima das 4 réplicas; com `maxUnavailable: 0` só derruba um antigo depois que o novo fica disponível. O intervalo entre o novo Pod ficar `1/1` (7s) e o antigo entrar em `Terminating` é o `minReadySeconds: 15`: o Pod precisa ficar pronto por 15s para contar como disponível.

Histórico e ReplicaSets após as 4 aplicações

```
$ kubectl -n store-dev get deploy store-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
store-api   4/4     4            4           4m40s

$ kubectl -n store-dev rollout history deployment/store-api
deployment.apps/store-api
REVISION  CHANGE-CAUSE
1         rev1: store-api:1.0, 4 replicas
2         rev2: imagem store-api:2.0
3         rev3: APP_ENV=homolog
4         rev4: limits.memory=384Mi

$ kubectl -n store-dev get rs -o wide
NAME                   DESIRED   CURRENT   READY   AGE     CONTAINERS   IMAGES          SELECTOR
store-api-5b5d55bb94   0         0         0       4m40s   store-api    store-api:1.0   app=store-
api,pod-template-hash=5b5d55bb94
store-api-656d8854b5   0         0         0       2m51s   store-api    store-api:2.0   app=store-
api,pod-template-hash=656d8854b5
store-api-7948549dc9   0         0         0       4m17s   store-api    store-api:2.0   app=store-
api,pod-template-hash=7948549dc9
store-api-85f69cbc57   4         4         4       86s     store-api    store-api:2.0   app=store-
api,pod-template-hash=85f69cbc57
```

Todas as revisões têm `CHANGE-CAUSE` preenchido e só uma ReplicaSet tem DESIRED 4.

Por que aparecem 4 revisões e não 3

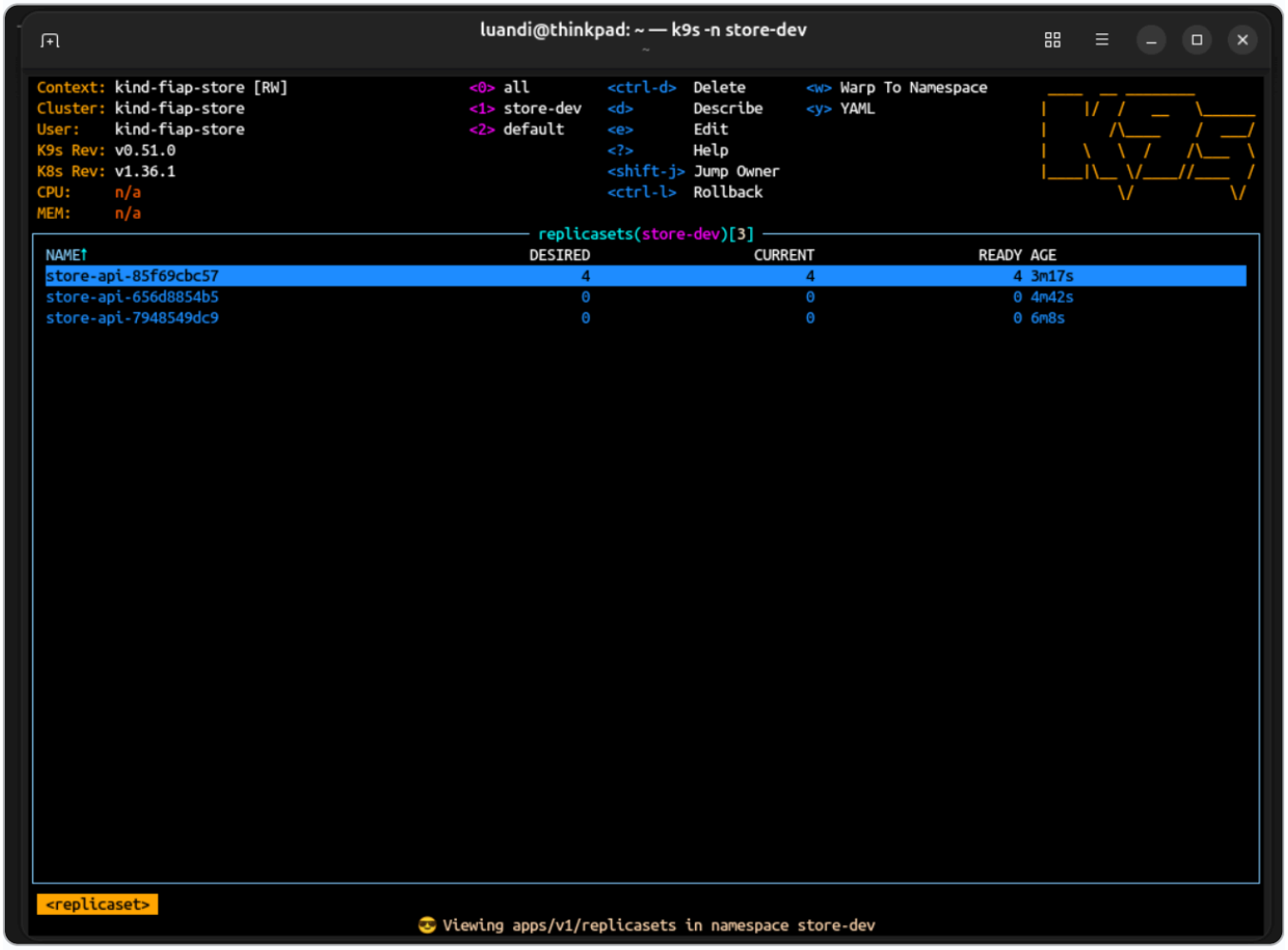
O enunciado espera 3 revisões visíveis com `revisionHistoryLimit: 3`, mas o resultado real foi 4. A documentação oficial define o campo como "the number of old ReplicaSets to retain": ele conta só as ReplicaSets **antigas**. Com 3, sobram 3 antigas + a atual = 4 revisões. Para ficar com exatamente 3, o limite precisa ser 2. Prova, sem criar revisão nova (o patch não altera o template do Pod):

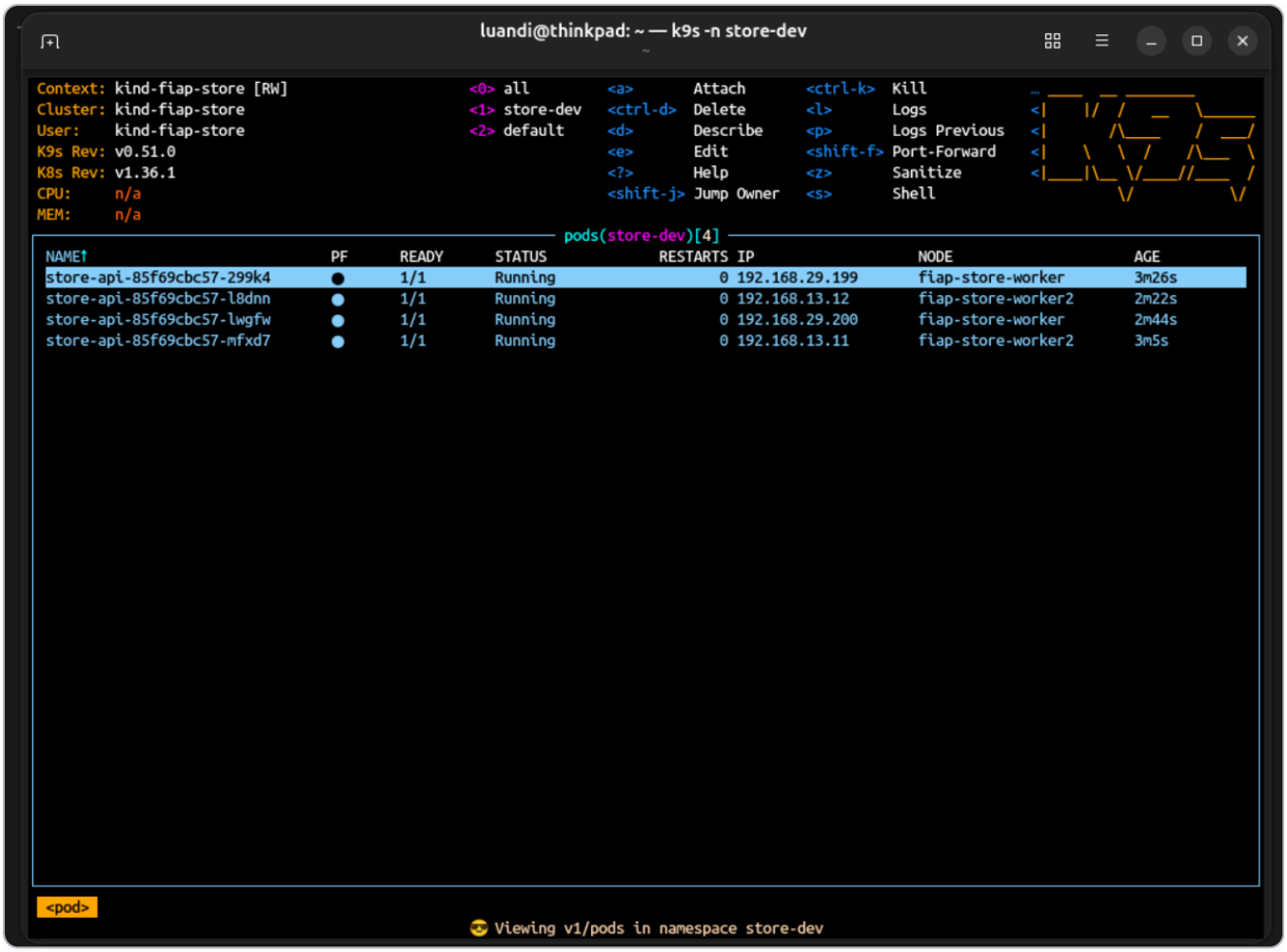
```
$ kubectl -n store-dev patch deployment/store-api -p '{"spec":{"revisionHistoryLimit":2}}'
deployment.apps/store-api patched

$ kubectl -n store-dev rollout history deployment/store-api
deployment.apps/store-api
REVISION  CHANGE-CAUSE
2         rev2: imagem store-api:2.0
3         rev3: APP_ENV=homolog
4         rev4: limits.memory=384Mi

$ kubectl -n store-dev get rs -o wide
NAME                   DESIRED   CURRENT   READY   AGE     CONTAINERS   IMAGES           SELECTOR
store-api-656d8854b5   0         0         0       3m25s   store-api    store-api:2.0    app=store-
api,pod-template-hash=656d8854b5
store-api-7948549dc9   0         0         0       4m51s   store-api    store-api:2.0    app=store-
api,pod-template-hash=7948549dc9
store-api-85f69cbc57   4         4         4       2m      store-api    store-api:2.0    app=store-
api,pod-template-hash=85f69cbc57
```

A ReplicaSet da revisão 1 (`store-api:1.0`) foi coletada e o histórico ficou com 3 revisões, atendendo o critério de aceite.




luandi@thinkpad: ~ — k9s -n store-dev
Context: kind-fiap-store [RW]
Cluster: kind-fiap-store
User: kind-fiap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a
<0> all
<1> store-dev
<2> default
<a> Attach
<ctrl-d> Delete
<d> Describe
<e> Edit
<?> Help
<shift-j> Jump Owner
<ctrl-k> Kill
<l> Logs
<p> Logs Previous
<shift-f> Port-Forward
<z> Sanitize
<s> Shell
pods(store-dev)[4]
NAME↑ PF READY STATUS RESTARTS IP NODE AGE
store-api-85f69cbc57-299k4 1/1 Running 0 192.168.29.199 fiap-store-worker 3m26s
store-api-85f69cbc57-l8dnn 1/1 Running 0 192.168.13.12 fiap-store-worker2 2m22s
store-api-85f69cbc57-lwgfw 1/1 Running 0 192.168.29.200 fiap-store-worker 2m44s
store-api-85f69cbc57-mfxd7 1/1 Running 0 192.168.13.11 fiap-store-worker2 3m5s
<pod>
Viewing v1/pods in namespace store-dev